Forelesning 4: Kalibrering og sammenlikning av resultater (løsningsforslag)

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy import stats
```

Repetisjon

Gruppering med `groupby`

```
penguins = pd.read_csv("../datahandtering_visualisering/data/penguins.csv")
penguins.head()
```

```
penguins.groupby("species")["body_mass_g"].mean()
```

```
penguins.groupby("species")["body_mass_g"].agg(["count", "mean", "std"])
```

```
flame = pd.read_csv("../datahandtering_visualisering/data/flame_emission.csv")

summary = flame.groupby("ion")["emission_589_nm"].agg(["count", "mean", "std"])
summary["rsd_percent"] = summary["std"] / summary["mean"] * 100
summary
```

Del 1: Kalibrering

Se på dataene først

```
data = pd.read_csv("../datahandtering_visualisering/data/uvvis_calibration.csv")
standards = data[data["sample_type"] != "unknown"]

plt.scatter(standards["concentration_uM"], standards["absorbance"])
plt.xlabel("Concentration (µmol/L)")
plt.ylabel("Absorbance")
plt.show()
```

Lineær regresjon

```
result = stats.linregress(standards["concentration_uM"], standards["absorbance"])

print(f"Slope: {result.slope:.5f} L/µmol")
print(f"Intercept: {result.intercept:.5f}")
print(f"R²: {result.rvalue**2:.6f}")
```

```
x_model = np.linspace(0, 10, 100)
y_model = result.slope * x_model + result.intercept

plt.scatter(standards["concentration_uM"], standards["absorbance"],
label="Measurements")
plt.plot(x_model, y_model, color="#b23a48", label="Linear model")
plt.xlabel("Concentration (µmol/L)")
plt.ylabel("Absorbance")
plt.legend()
plt.show()
```

Bestem den ukjente konsentrasjonen

```
unknown = data[data["sample_type"] == "unknown"]
A_unknown = unknown["absorbance"].mean()

c_unknown = (A_unknown - result.intercept) / result.slope
print(f"Concentration: {c_unknown:.2f} µmol/L")
```

```
shifted = stats.linregress(standards["concentration_uM"], standards["absorbance"]
+ 0.01)
print(f"Slope: {shifted.slope:.5f}   Intercept: {shifted.intercept:.5f}")
```

Del 2: Stemmer resultatet med en referanseverdi?

```
iron_mg_L = np.array([5.12, 5.08, 5.15, 5.10, 5.13])

ci = stats.t.interval(0.95, len(iron_mg_L) - 1, loc=np.mean(iron_mg_L),
scale=stats.sem(iron_mg_L))
print(ci)
```

Hypotesetesting

```
test = stats.ttest_1samp(iron_mg_L, popmean=5.00)
print(f"t = {test.statistic:.2f}   p = {test.pvalue:.5f}")
```

```
measurements = np.array([99.8, 100.3, 100.1, 99.9, 100.4])

ci = stats.t.interval(0.95, len(measurements) - 1, loc=np.mean(measurements),
scale=stats.sem(measurements))
test = stats.ttest_1samp(measurements, popmean=100.0)

print(ci)
print(f"p = {test.pvalue:.3f}")
```

Del 3: To uavhengige grupper

```
lake_A = np.array([2.31, 2.45, 2.38, 2.52, 2.41, 2.36])
lake_B = np.array([2.68, 2.74, 2.61, 2.79, 2.70, 2.66])

test = stats.ttest_ind(lake_A, lake_B, equal_var=False)
print(f"Difference: {np.mean(lake_B) - np.mean(lake_A):.2f} mg/L   p =
{test.pvalue:.5f}")
```

```
catalyst_A = np.array([78.2, 79.1, 77.8, 80.0, 78.6])
catalyst_B = np.array([81.3, 80.7, 82.1, 81.5, 80.9])

test = stats.ttest_ind(catalyst_A, catalyst_B, equal_var=False)
print(f"Difference: {np.mean(catalyst_B) - np.mean(catalyst_A):.2f} %   p =
{test.pvalue:.4f}")
```

Del 4: To analysemetoder på de samme prøvene

```
methods =
pd.read_csv("../datahandtering_visualisering/data/method_comparison.csv")
methods["difference"] = methods["method_B_mg_L"] - methods["method_A_mg_L"]
methods
```

```
test = stats.ttest_rel(methods["method_B_mg_L"], methods["method_A_mg_L"])
print(f"p = {test.pvalue:.6f}")
```

Del 5: En mistenkelig måling

```
def grubbs(measurements, alpha=0.05):
    """Returnerer beregnet og kritisk G for Grubbs' test (tosidig)."""
    n = len(measurements)
    G = np.max(np.abs(measurements - np.mean(measurements))) /
np.std(measurements, ddof=1)
    t_crit = stats.t.ppf(1 - alpha / (2 * n), n - 2)
    G_crit = (n - 1) / np.sqrt(n) * np.sqrt(t_crit**2 / (n - 2 + t_crit**2))
    return G, G_crit
```

```
series = np.array([10.02, 10.05, 9.98, 10.01, 10.42])

G, G_crit = grubbs(series)
print(f"G = {G:.3f}   G_crit = {G_crit:.3f}")
```

```
without = series[series != 10.42]

print(f"With:    mean = {np.mean(series):.3f}   SD = {np.std(series,
ddof=1):.3f}")
print(f"Without: mean = {np.mean(without):.3f}   SD = {np.std(without,
ddof=1):.3f}")
```

Oppgaver

Oppgave 1

```
print(f"A = {result.slope:.4f} · c + {result.intercept:.4f}   (R² =
{result.rvalue**2:.5f})")
print(f"Concentration: {c_unknown:.2f} µmol/L")
```

Oppgave 3

```
changed = standards.copy()
changed.loc[changed["absorbance"] == 0.809, "absorbance"] = 0.700

result_changed = stats.linregress(changed["concentration_uM"],
changed["absorbance"])
c_changed = (A_unknown - result_changed.intercept) / result_changed.slope

print(f"Slope: {result_changed.slope:.5f}   R²: {result_changed.rvalue**2:.4f}
Concentration: {c_changed:.2f} µmol/L")
```

Oppgave 4

```
plt.plot(methods["difference"], "o")
plt.axhline(methods["difference"].mean(), color="gray", linestyle="--")
plt.xlabel("Sample number")
plt.ylabel("Method B − method A (mg/L)")
plt.show()
```